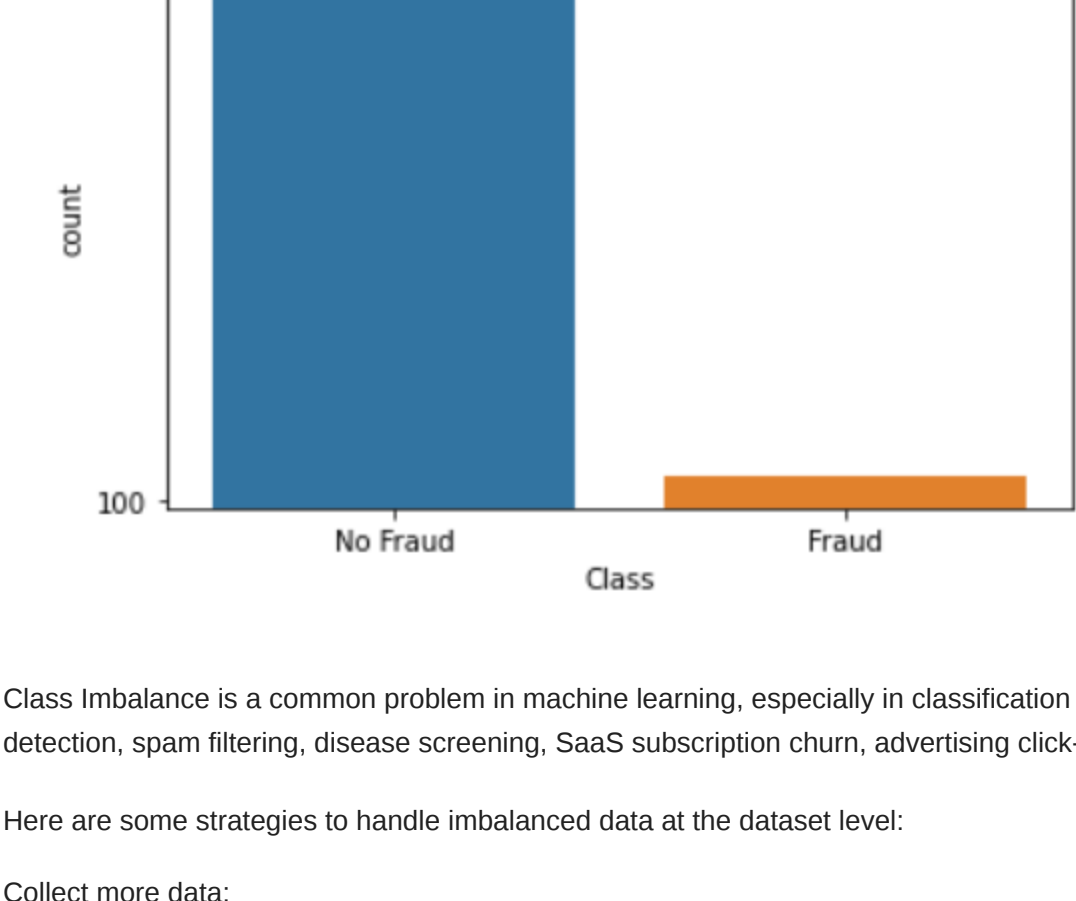


Techniques to Solve Imbalanced Classes in Machine Learning

While working as a data scientist, some of the most frequently occurring problem statements are related to binary classification. A common problem when solving these problem statements is that of class imbalance. When observation in one class is higher than in other classes, a class imbalance exists. Example: To detect fraudulent credit card transactions. As shown in the graph below, the fraudulent transaction is around 400 compared to the non-fraudulent transaction of around 90000.



Class imbalance is a common problem in machine learning, especially in classification problems. Imbalance data can hamper our model accuracy big time. It appears in many domains, including fraud detection, spam filtering, disease screening, SaaS subscription churn, advertising click-throughs, etc.

Here are some strategies to handle imbalanced data at the dataset level:

Collect more data:

Gathering additional data, especially from the minority class, can help improve the model's ability to learn patterns and make accurate predictions. More data provides the model with a better understanding of the underlying distribution.

Data augmentation:

Data augmentation techniques involve creating synthetic samples by applying various transformations to existing samples in the minority class. This can help balance the dataset and provide more training examples for the minority class.

Subset sampling:

Instead of using the entire dataset, you can randomly select a subset of the majority class samples to match the size of the minority class. This approach reduces the dominance of the majority class and creates a more balanced dataset.

Cluster-based sampling:

Clustering techniques can be applied to identify groups within the majority class. You can then select representative samples from each cluster to create a more balanced dataset.

One-class classification:

In some cases, you may only have data from one class, making it challenging to create a balanced dataset. One-class classification techniques aim to model and identify instances that belong to a specific class while considering all other instances as outliers or anomalies.

Data cleaning:

Inspect the dataset for any mislabeled or incorrectly labeled samples. Removing such errors can improve the overall quality of the dataset and potentially address class imbalance to some extent.

Stratified sampling:

When splitting the dataset into training and testing subsets, use stratified sampling to ensure that the distribution of classes remains consistent in both subsets. This helps to maintain the representation of minority and majority classes in the evaluation phase.

Remember, these strategies for handling imbalanced data at the dataset level are not mutually exclusive. You can combine multiple techniques based on the characteristics of your dataset and the specific requirements of your problem. Additionally, it is essential to evaluate the impact of these techniques on the model's performance using appropriate evaluation metrics to ensure the effectiveness of the chosen approach.

The Problem With Class Imbalance in Machine Learning

Most machine learning algorithms work best when the number of samples in each class is about equal. This is because most algorithms are designed to maximize accuracy and reduce errors.

However, if the dataframe has imbalanced classes, then in such cases, you get a pretty high accuracy just by predicting the majority class, but you fail to capture the minority class, which is most often the point of creating the model in the first place. For example, if the class distribution shows that 99% of the data has the majority class, then any basic classification model like the logistic regression or decision tree will not be able to identify the minor class data points.

Credit Card Fraud Detection Example

Let's say we have a dataset of credit card companies where we have to find out whether the credit card transaction was fraudulent or not. But here's the catch... fraud transaction is relatively rare. Only 6% of the transactions are fraudulent.

Unfortunately, that accuracy is misleading.

For all those non-fraudulent transactions, you'd have 100% accuracy. For those transactions which are fraudulent, you'd have 0% accuracy. Your overall accuracy would be high simply because most of the transactions are not fraudulent (not because your model is any good).

This is clearly a problem because many machine learning algorithms are designed to maximize overall accuracy.

The Metric Trap

One of the major issues that new developer users fall into when dealing with unbalanced datasets relates to the evaluation metrics used to evaluate their machine learning model. Using simpler metrics like accuracy score can be misleading. In a dataset with highly unbalanced classes, the classifier will always "predicts" the most common class without performing any analysis of the features, and it will have a high accuracy rate, obviously not the correct one.

```
In [ ]: #Let's do this experiment using the simple XGBClassifier and no feature engineering:
```

```
# import library
```

```
# from xgboost import XGBClassifier
```

```
# xgb_model = XGBClassifier().fit(x_train, y_train)
```

```
# predict
```

```
# xgb_y_predict = xgb_model.predict(x_test)
```

```
# accuracy score
```

```
# xgb_score = accuracy_score(xgb_y_predict, y_test)
```

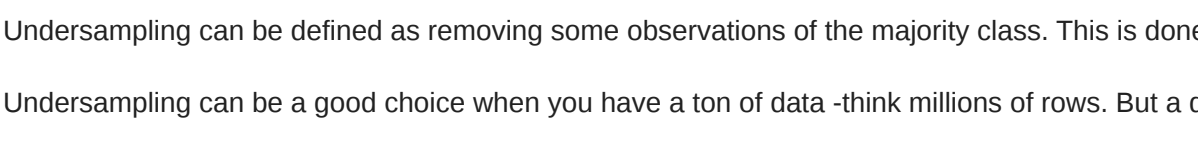
```
# print('Accuracy score is:', xgb_score)OUTPUT
```

```
# Accuracy score is: 0.992
```

We can see 99% accuracy, we are getting very high accuracy because it is predicting mostly the majority class that is 0 (Non-fraudulent).

Resampling Techniques to Solve Class Imbalance

One of the widely adopted class imbalance techniques for dealing with highly unbalanced datasets is called resampling. It consists of removing samples from the majority class (under-sampling) and/or adding more examples from the minority class (over-sampling).



Despite the advantage of balancing classes, these techniques also have their weaknesses (there is no free lunch).

The simplest implementation of over-sampling is to duplicate random records from the minority class, which can cause overfitting.

In under-sampling, the simplest technique involves removing random records from the majority class, which can cause a loss of information.

```
In [1]: # Let's implement this with the credit card fraud detection example.
```

```
# We will start by separating the class that will be 0 and class 1.
```

```
# class count
```

```
# class_count_0, class_count_1 = data['Class'].value_counts()
```

```
# Separate class
```

```
# class_0 = data[data['Class'] == 0]
```

```
# class_1 = data[data['Class'] == 1]# print the shape of the class
```

```
# print('class 0:', class_0.shape)
```

```
# print('class 1:', class_1.shape)
```

1. Random Under-Sampling

Under-sampling can be defined as removing some observations of the majority class. This is done until the majority and minority class is balanced out.

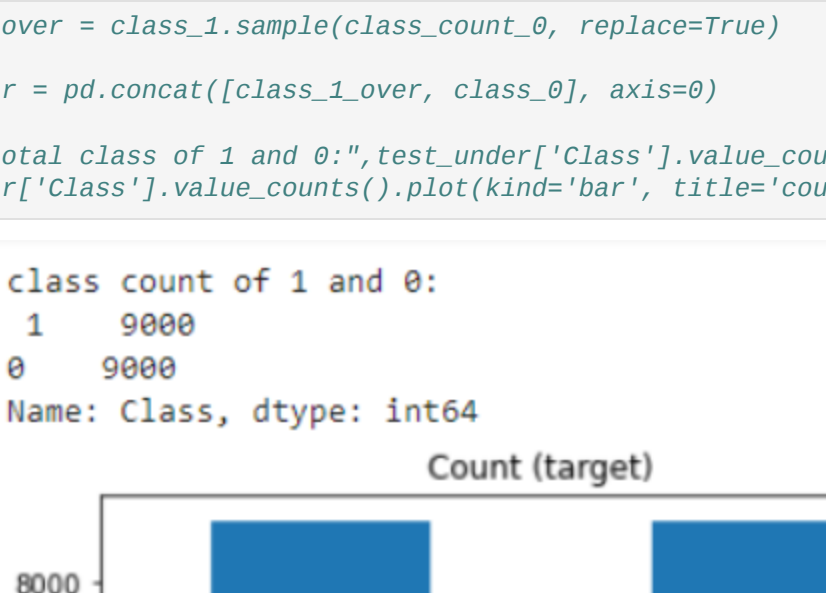
Under-sampling can be a good choice when you have a ton of data -think millions of rows. But a drawback to under-sampling is that we are removing information that may be valuable.

```
In [ ]: # class_0_under = class_0.sample(class_count_1)
```

```
# test_under = pd.concat([class_0_under, class_1], axis=0)
```

```
# print("total class of 1 and 0:",test_under['Class'].value_counts())# plot the count after under-sampling
```

```
# test_under['Class'].value_counts().plot(kind='bar', title='count (target)')
```



2. Random Over-Sampling

Oversampling can be defined as adding more copies to the minority class. Oversampling can be a good choice when you don't have a ton of data to work with.

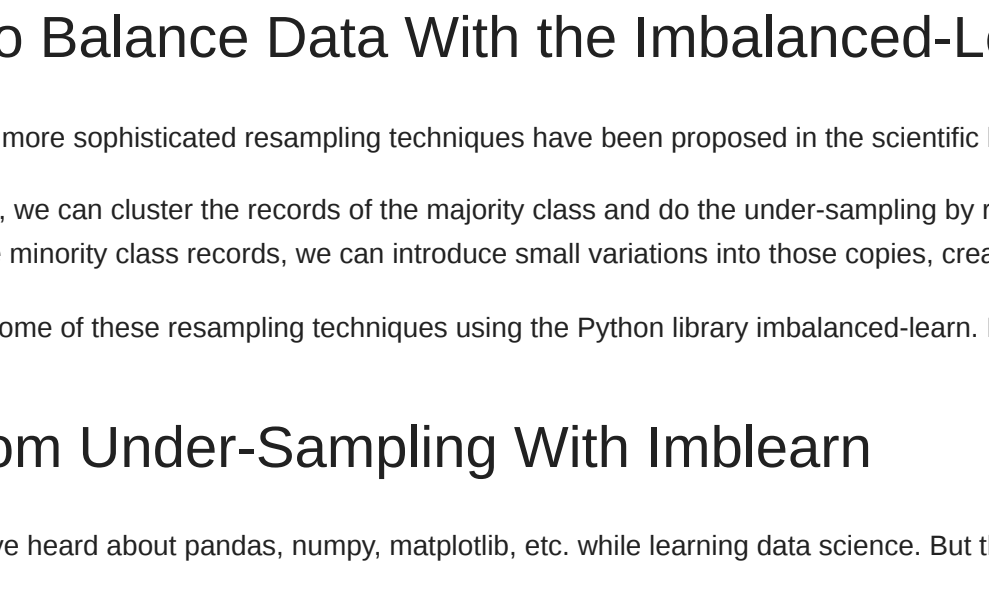
A con to consider when under-sampling is that it can cause overfitting and poor generalization to your test set.

```
In [3]: # class_1_over = class_1.sample(class_count_0, replace=True)
```

```
# test_over = pd.concat([class_1_over, class_0], axis=0)
```

```
# print("total class of 1 and 0:",test_over['Class'].value_counts())# plot the count after under-sampling
```

```
# test_over['Class'].value_counts().plot(kind='bar', title='count (target)')
```



How to Balance Data With the Imbalanced-Learn Python Module?

A number of more sophisticated resampling techniques have been proposed in the scientific literature.

For example, we can cluster the records of the majority class and do the under-sampling by removing records from each cluster, thus seeking to preserve information. In over-sampling, instead of creating exact copies of the minority class records, we can introduce small variations into those copies, creating more diverse synthetic samples.

Let's apply some of these resampling techniques using the Python library imbalanced-learn. It is compatible with scikit-learn and is part of scikit-learn-contrib projects.

Random Under-Sampling With Imblearn

You may have heard about pandas, numpy, matplotlib, etc. while learning data science. But there is another library: imblearn, which is used to sample imbalanced datasets and improve your model performance.

RandomUnderSampler is a fast and easy way to balance the data by randomly selecting a subset of data for the targeted classes. Under-sample the majority class(es) by randomly picking samples with or without replacement.

```
In [6]: # import library
```

```
# from imblearn.under_sampling import RandomUnderSampler
```

```
# rus = RandomUnderSampler(random_state=42, replacement=True)# fit predictor and target variable
```

```
# x_rus, y_rus = rus.fit_resample(x, y)
```

```
# print('original dataset shape:', Counter(y))
```

```
# print('Resample dataset shape', Counter(y_rus))
```

```
original dataset shape: Counter({0: 9000, 1: 492})
```

```
Resample dataset shape Counter({0: 492, 1: 492})
```

Random Over-Sampling With imblearn

One way to fight imbalanced data is to generate new samples in the minority classes. The most naive strategy is to generate new samples by random sampling with the replacement of the currently available samples. The RandomOverSampler offers such a scheme.

```
In [ ]: # import library
```

```
# from imblearn.over_sampling import RandomOverSampler
```

```
# ros = RandomOverSampler(random_state=42)
```

```
# fit predictor and target variablex_ros, y_ros = ros.fit_resample(x, y)
```

```
# print('Original dataset shape', Counter(y))
```

```
# print('Resample dataset shape', Counter(y_ros))
```

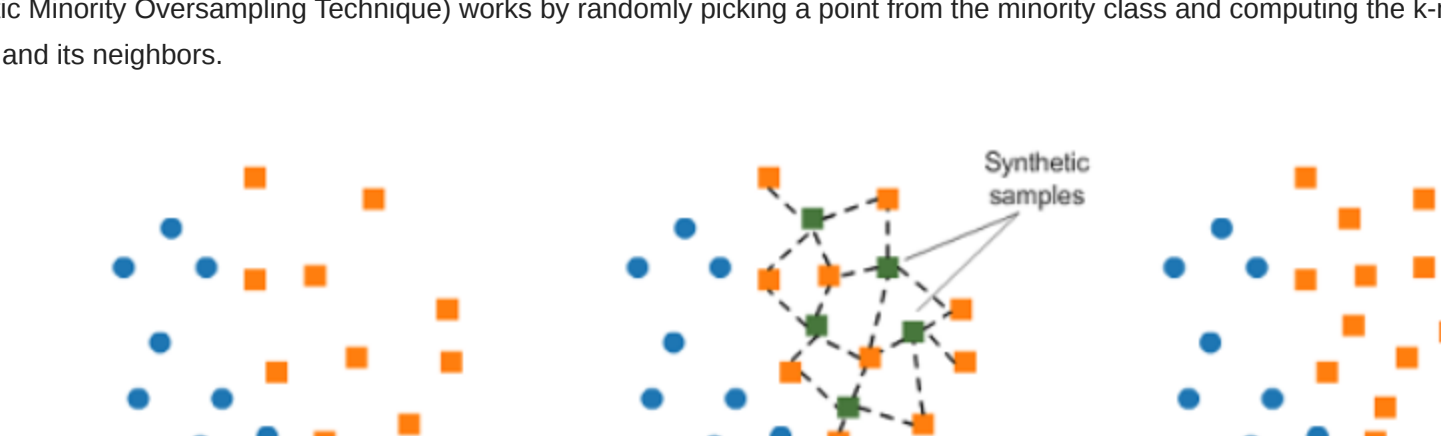
```
Original dataset shape Counter({0: 9000, 1: 492})
```

```
Resample dataset shape Counter({1: 9000, 0: 9000})
```

Under-Sampling: Tomek Links

Tomek links are pairs of very close instances but of opposite classes. Removing the instances of the majority class of each pair increases the space between the two classes, facilitating the classification process.

Tomek's link exists if the two samples are the nearest neighbors of each other.



```
In [ ]: # In the code below, we'll use ratio='majority' to resample the majority class.
```

```
# import library
```

```
# from imblearn.under_sampling import TomekLinks
```

```
# tl = RandomOverSampler(sampling_strategy='majority')
```

```
# fit predictor and target variable
```

```
# x_tl, y_tl = ros.fit_resample(x, y)
```

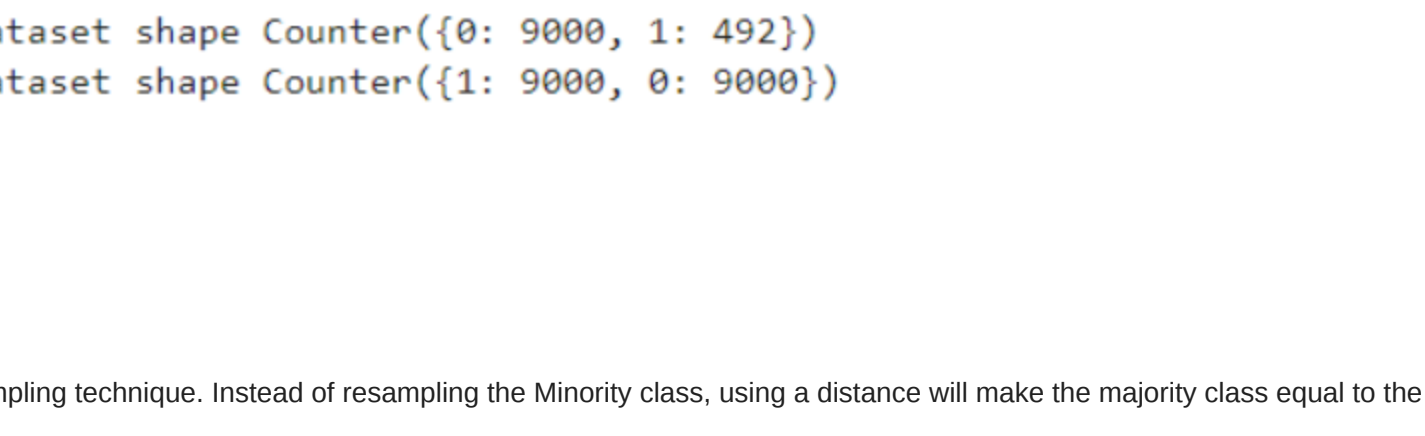
```
# print('Original dataset shape', Counter(y))
```

```
# print('Resample dataset shape', Counter(y_ros))
```

Synthetic Minority Oversampling Technique (SMOTE)

This technique generates synthetic data for the minority class.

SMOTE (Synthetic Minority Oversampling Technique) works by randomly picking a point from the minority class and computing the k-nearest neighbors for this point. The synthetic points are added between the chosen point and its neighbors.



SMOTE algorithm works in 4 simple steps:

Choose a minority class as the input vector.

Find its k nearest neighbors (k_neighbors is specified as an argument in the SMOTE() function).

Choose one of these neighbors and place a synthetic point anywhere on the line joining the point under consideration and its chosen neighbor.

Repeat the steps until the data is balanced.

```
In [ ]: # import library
```

```
# from imblearn.over_sampling import SMOTE
```

```
# smote = SMOTE()
```

```
# fit predictor and target variable
```

```
# x_smote, y_smote = smote.fit_resample(x, y)
```

```
# print('Original dataset shape', Counter(y))
```

```
# print('Resample dataset shape', Counter(y_ros))
```

```
Original dataset shape Counter({0: 9000, 1: 492})
```

```
Resample dataset shape Counter({0: 492, 1: 492})
```

NearMiss

NearMiss is an under-sampling technique. Instead of resampling the Minority class, using a distance will make the majority class equal to the minority class.

```
In [ ]: # from imblearn.under_sampling import NearMiss
```

```
# nm = NearMiss()
```

```
# x_nm, y_nm = nm.fit_resample(x, y)
```

```
# print('Original dataset shape:', Counter(y))
```

```
# print('Resample dataset shape:', Counter(y_nm))
```

```
Original dataset shape: Counter({0: 16000, 1: 492})
```

```
Resample dataset shape: Counter({0: 492, 1: 492})
```

Advantages and Disadvantages of Under-Sampling

Advantage:

It can help improve run time and storage problems by reducing the number of training data samples when the training data set is huge.

Disadvantages:

It can discard potentially useful information which could be important for building rule classifiers.

The sample chosen by random under-sampling may be a biased sample. And it will not be an accurate representation of the population. Thereby resulting in inaccurate results with the actual test data set.

Advantages and Disadvantages of Over-Sampling

Advantages:

Unlike under-sampling, this method leads to no information loss.

Outperforms under sampling

Disadvantages:

It increases the likelihood of overfitting since it replicates the minority class events.

```
In [ ]:
```

```
In [ ]:
```